SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Submission Template

Student Name	Tharun
Register Number	1924260280
Course Code & Name	Advanced Data Structures and Design Engineering
Slot	B
Assignment / Task Title	Structures, Unions, Storage Classes and Design — Tasks 1 to 6 (50 Marks)
GitHub Repository URL	
Submission Date	11/11/2026

Instructions to Students

1. Complete every section for each assigned task.
2. Write your own explanation, algorithm/pseudocode, and program.
3. Run the program and insert a clear screenshot of the output.
4. Include test cases and a short analysis where applicable.
5. If code is maintained on GitHub, provide the correct repository link.

TASK 1 — Data Design Review — Diagnose and Repair a Poor Data Design

Task / Problem Statement

A junior developer wrote the data part of a primary health centre's patient register using five parallel arrays (`name, age, phone, bp, visitDate`) and a global `count`, with `addPatient()` taking five loose parameters. The task is to (1.1) find at least five design deficiencies and say what could go wrong in the health centre, (1.2) redesign the data with structures (nested structures for the date and the blood pressure) and rewrite `addPatient()` to accept a structure pointer, and (1.3) defend the new design in not more than 100 words. The alternative concept (parallel arrays vs an array of structures) is also analysed for time and space.

Aim

To diagnose the weaknesses of a parallel-array design and repair it with nested structures and pointer-based functions, so that every patient record is kept together, validated and easy to sort.

Concept / Theory

A **structure** groups related data of different types under one name, so one patient is one variable. A structure can contain another structure (**nesting**), which suits values with parts, such as a date (day, month, year) or a blood pressure (systolic, diastolic). A **structure pointer** with the `->` operator lets a function work on the caller's record without copying it, and one structure assignment copies a whole record. In an **array of structures** every element holds all fields of one patient, while parallel arrays keep each field in a separate array tied together only by the index. The compiler adds **padding** inside structures to keep members aligned, which affects `sizeof`.

Task 1.1 — Diagnosis of the original design

#	Deficiency	What could go wrong in the health centre
1	Related data kept in parallel arrays	A patient exists only as the same index in five arrays. If one array is changed, sorted or deleted without the others, a name gets another patient's age and blood pressure, which can lead to wrong treatment.
2	Phone number of 10 digits in `char[10]`	There is no room for the terminating `'\0'`; `strcpy` writes 11 bytes and spills into the next patient's phone number. Numbers get corrupted and patients cannot be reached. No digit or length check either.
3	Blood pressure (two values) in one `float`	`120/80` is not a number: a float cannot hold systolic and diastolic separately, so high-BP patients cannot be identified.
4	Date stored as text of 8 characters	`10/11/26` needs 9 bytes with `'\0'`, so it overflows. Text cannot be compared or subtracted, so follow-up dates cannot be calculated, and a two-digit year is ambiguous.
5	No limit check on `count`	The 101st patient writes beyond the end of all five arrays, corrupting memory or crashing the program and losing the register.
6	Global variables	Any function can change `count` or the arrays. Two registers (two clinics) cannot exist, and the code is hard to test and debug.
7	Difficult to sort by age	Sorting `age[]` alone breaks the link to the other arrays; all five must be swapped together (strings need `strcpy`). Missing one array silently mismatches the records.

Procedure / Algorithm

1. Study the original arrays and list the design problems (Task 1.1).
2. Define `struct date` and `struct bp` so that the date and the blood pressure become nested structures.
3. Define `struct patient` with name, age, `phone[11]` (10 digits + `'\0'`), pressure and visit date.
4. Define `struct patient_register` holding `list[100]` and `count`, so no global variable is needed.
5. Write `addPatient(struct patient_register *reg, const struct patient *p)` with a limit check and a phone-length check; return 0 or -1.
6. Write `sortByAge()` (selection sort that swaps whole structures) and `printPatient()`.
7. Compile and run with valid patients, an invalid phone number and a full register; compare the memory used by both designs.

Program / Source Code

task1.c

```
#include <stdio.h>
#include <string.h>

#define MAX_PATIENTS 100

struct date { int day, month, year; };
struct bp   { int systolic, diastolic; };

struct patient {
    char name[30];
    int age;
    char phone[11];          /* 10 digits + '\0' */
    struct bp pressure;      /* nested: 120/80 */
    struct date visit;       /* nested: dd/mm/yyyy */
};

struct patient_register {
    struct patient list[MAX_PATIENTS];
    int count;               /* no global variable */
};

/* returns 0 on success, -1 if the register is full or the input is bad */
int addPatient(struct patient_register *reg, const struct patient *p)
{
    if (reg->count >= MAX_PATIENTS) return -1;      /* limit check */
    if (strlen(p->phone) != 10)     return -1;      /* phone check */
    reg->list[reg->count] = *p;                     /* one assignment copies the record */
    reg->count++;
    return 0;
}

void sortByAge(struct patient_register *reg)        /* selection sort */
{
    for (int i = 0; i < reg->count - 1; i++) {
        int min = i;
        for (int j = i + 1; j < reg->count; j++)
            if (reg->list[j].age < reg->list[min].age) min = j;
        if (min != i) {
            struct patient t = reg->list[i];
            reg->list[i] = reg->list[min];
            reg->list[min] = t;
        }
    }
}

void printPatient(const struct patient *p)
{
    printf("%-8s age %2d  ph %s  BP %d/%d  visit %02d/%02d/%d\n",
           p->name, p->age, p->phone, p->pressure.systolic,
           p->pressure.diastolic, p->visit.day, p->visit.month, p->visit.year);
}

int main(void)
{
    struct patient_register reg = { .count = 0 };
    struct patient in[] = {
        {"Ravi",   54, "9876543210", {140, 90}, {10, 11, 2026}},
        {"Meena",  23, "9123456780", {118, 76}, {10, 11, 2026}},
        {"Kumar",  67, "9000011111", {150, 95}, {11, 11, 2026}},
        {"Anu",     8, "98765",      {100, 60}, {11, 11, 2026}}   /* bad phone */
    };
    for (int i = 0; i < 4; i++)
        printf("add %-6s -> %s\n", in[i].name,
               addPatient(&reg, &in[i]) == 0 ? "OK" : "REJECTED");

    sortByAge(&reg);
    printf("\nSorted by age:\n");
    for (int i = 0; i < reg.count; i++) printPatient(&reg.list[i]);
```

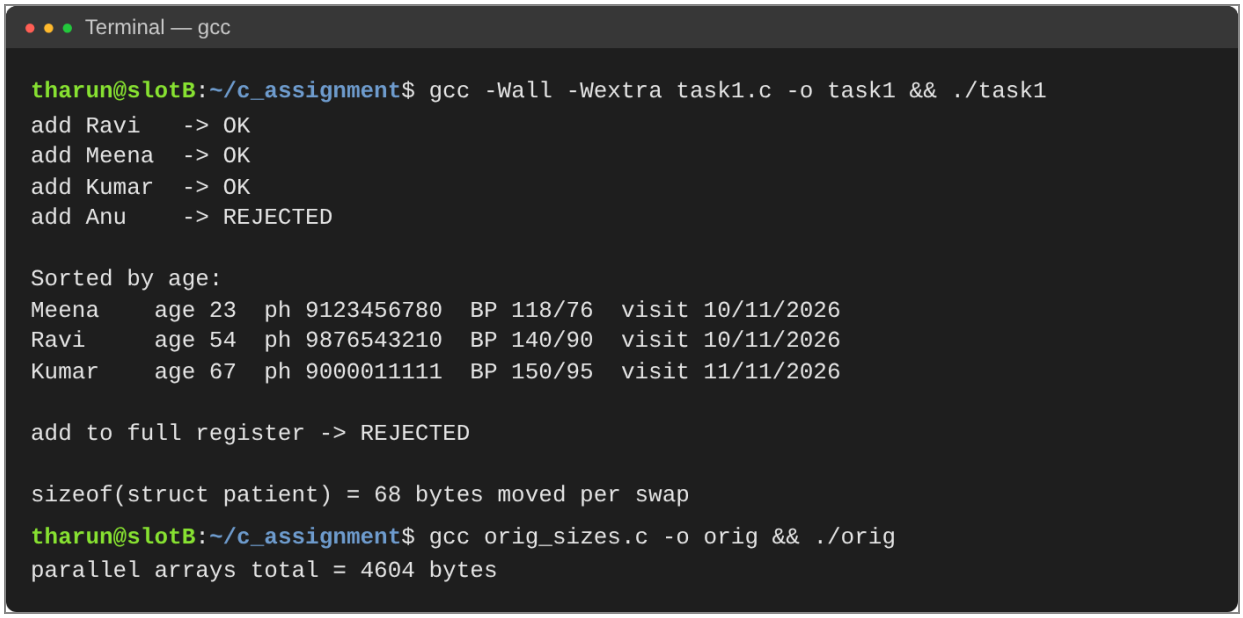
```
    /* full register test */
    struct patient_register full = { .count = MAX_PATIENTS };
    printf("\nadd to full register -> %s\n",
           addPatient(&full, &in[0]) == 0 ? "OK" : "REJECTED");

    printf("\nsizeof(struct patient) = %zu bytes moved per swap\n", sizeof(struct patient));
    return 0;
}
```

orig_sizes.c — size of the original parallel arrays

```
#include <stdio.h>
char name[100][20]; int age[100]; char phone[100][10]; float bp[100]; char visitDate[100][8]; int count;
int main(void){ printf("parallel arrays total = %zu bytes\n", sizeof name+sizeof age+sizeof phone+sizeof
bp+sizeof visitDate+sizeof count); return 0; }
```

Output Screenshot



Analysis / Discussion

Three valid patients were added and the one with a 5-digit phone number was rejected by the validation. After `sortByAge()` the records appear as 23, 54, 67 with all fields still together, which was the weakness of the parallel arrays. Adding to a register that already holds 100 patients is refused instead of overflowing.

Task 1.3 — Design defence (80 words)

Each patient is one structure, so name, age, phone, pressure and date always travel together. Sorting needs a single whole-record assignment, so no field is left behind and records never get mismatched. A function takes one pointer instead of five loose arrays and a global count, and can validate input. New data such as blood group needs only one new member, and existing code keeps working. Nested date and pressure structures can be compared, printed and reused on their own.

Alternative concept and complexity analysis

When parallel arrays are still preferred: when a loop uses only one field (for example the average age scans only `age[]`), because the data is contiguous, cache-friendly and easy for the compiler to vectorise; also when fields are added or processed column by column, and when padding must be avoided.

Time to sort n patients: selection sort does O(n²) comparisons and at most n − 1 swaps in both designs. In the array of structures a swap moves a whole record: 3 copies × `sizeof(struct patient)` = 3 × 68 = 204 bytes, in three assignments. In the parallel arrays a swap must be repeated on five arrays (five temporaries, and `strcpy` for

the strings), moving less data but needing more code that is easy to get wrong. Sorting an index array instead (4 bytes per swap) reduces the movement in both designs.

Space: the structure is 68 bytes: members total 30 + 4 + 11 + 8 + 12 = 65 bytes, plus 3 bytes of padding (2 after `name`, 1 after `phone`). For 100 patients that is 6,800 bytes. The original arrays take 4,604 bytes (measured), which is smaller only because they store less: shorter names, a float instead of two pressure values, text dates, and fields with no room for `'\0'`. Parallel arrays of one type each have almost no padding; the structure design spends about 3 bytes per record on padding to be correct and safe.

Result

The five-array design was diagnosed with seven deficiencies. It was repaired with a nested structure design that validates input, checks the limit and sorts without mismatching data. I learnt that grouping related data in a structure and passing a pointer to it is safer and easier to extend than parallel arrays.

TASK 2 — From Requirement to Nested Structures — Courier Parcel Tracking

Task / Problem Statement

A courier franchise books parcels at its counter. Each parcel has an ID, weight, booking date, sender and receiver (each with name, city and PIN code), a service type (normal or express) and a charge. The franchise needs a list of parcels sorted by weight and a list of parcels for a given city. The task is to (2.1) identify the entities and nested structures, (2.2) write the structure definitions with `typedef` and an array for the day's bookings, (2.3) write pseudocode for sorting and for listing by receiver city, (2.4) write the access paths for the receiver's PIN code of the third parcel, and complete TODO 1 and TODO 2 of the given program.

Aim

To convert a written requirement into entities and nested structures, and to use an array of structures with the dot and arrow operators to sort and search parcels.

Concept / Theory

Entity identification: nouns in the requirement with their own data become structures; a group of fields that appears twice (sender and receiver) becomes one reusable structure type. **Nested structures** place one structure inside another and are accessed by chaining the dot operator (`list[2].receiver.pincode`); through a pointer the first step uses `->` (`p->receiver.pincode`). `typedef` gives a structure a short type name. An **array of structures** stores the day's bookings, and a structure can be swapped with one assignment.

Procedure / Algorithm

Task 2.1 — Entities and nesting

Entity (structure)	Fields	Nested inside
Date	day, month, year	Parcel (booked)
Person (sender / receiver)	name, city, pincode	Parcel — twice: sender and receiver
Service (enumeration)	NORMAL, EXPRESS	Parcel (service)
Parcel	id, weight, booked (Date), sender (Person), receiver (Person), service, charge	— (array `bookings[]` holds the day's parcels)

Task 2.3 — Pseudocode

```
(a) SORT BY WEIGHT (selection sort, ascending)
  FOR i = 0 TO n-2
      min = i
      FOR j = i+1 TO n-1
          IF parcel[j].weight < parcel[min].weight THEN min = j
      IF min != i THEN swap whole structures parcel[i] and parcel[min]

(b) LIST PARCELS FOR A RECEIVER CITY
  READ city
  found = 0
  FOR i = 0 TO n-1
      IF parcel[i].receiver.city equals city THEN print parcel[i]; found = found + 1
  IF found = 0 THEN print "no parcel for that city"
```

Task 2.4 — Access paths (third parcel = index 2)

(i) Array and dot operator: `bookings[2].receiver.pincode`
(ii) Pointer and arrow operator, with `Parcel *p = bookings;`: `(p + 2)->receiver.pincode` (or `p[2].receiver.pincode`)

Program / Source Code

task2_design.c — Task 2.2 structure definitions and access paths

```
#include <stdio.h>

typedef struct { int day, month, year; } Date;
typedef struct { char name[20]; char city[20]; int pincode; } Person;
typedef enum { NORMAL, EXPRESS } Service;
typedef struct {
    int     id;
    float   weight;
    Date    booked;
    Person  sender;
    Person  receiver;
    Service service;
    float   charge;
} Parcel;

Parcel bookings[100];          /* the day's bookings */
int    nbookings = 0;

int main(void)
{
    Parcel p3 = { 503, 2.8f, {13,10,2026}, {"Arun","Madurai",625001},
                  {"Bala","Coimbatore",641001}, EXPRESS, 85.0f };
    bookings[nbookings++] = p3; bookings[nbookings++] = p3; bookings[nbookings++] = p3;
    Parcel *p = bookings;
    printf("(i)  bookings[2].receiver.pincode      = %d\n", bookings[2].receiver.pincode);
    printf("(ii) (p+2)->receiver.pincode           = %d\n", (p + 2)->receiver.pincode);
    printf("sizeof(Date)=%zu sizeof(Person)=%zu sizeof(Parcel)=%zu\n",
           sizeof(Date), sizeof(Person), sizeof(Parcel));
    return 0;
}
```

task2.c — completed program (TODO 1 and TODO 2)

```
#include <stdio.h>

struct date    { int day, month, year; };
struct address { char city[20]; int pincode; };
struct parcel  { int id; float weight; struct address to; struct date booked; };

/* flat version for the complexity comparison */
struct parcel_flat { int id; float weight; char city[20]; int pincode;
                     int day, month, year; };

void print_parcel(const struct parcel *p)
{
    printf("%d %.1f kg %s-%d %02d/%02d/%d\n", p->id, p->weight, p->to.city,
           p->to.pincode, p->booked.day, p->booked.month, p->booked.year);
}

/* Selection sort of the parcels in ascending order of weight. */
void sort_by_weight(struct parcel p[], int n)
{
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (p[j].weight < p[min].weight) min = j;
        if (min != i) {
            struct parcel t = p[i];
            p[i] = p[min];
            p[min] = t;
        }
    }
}

int main(void)
{
    struct parcel list[3] = {
        {501, 4.5f, {"Madurai", 625001},    {12, 10, 2026}},
```

```
        {502, 1.2f, {"Chennai", 600077},    {12, 10, 2026}},
        {503, 2.8f, {"Coimbatore", 641001}, {13, 10, 2026}}
    };
    sort_by_weight(list, 3);
    for (int i = 0; i < 3; i++) print_parcel(&list[i]);
    printf("bytes copied per swap = 3 x sizeof(struct parcel) = 3 x %zu = %zu\n",
           sizeof(struct parcel), 3 * sizeof(struct parcel));
    printf("sizeof nested = %zu, sizeof flat = %zu\n",
           sizeof(struct parcel), sizeof(struct parcel_flat));
    return 0;
}
```

Output Screenshot

Terminal — gcc

```
tharun@slotB:~/c_assignment$ gcc -Wall -Wextra task2_design.c -o d && ./d
(i)  bookings[2].receiver.pincode      = 641001
(ii) (p+2)->receiver.pincode           = 641001
sizeof(Date)=12 sizeof(Person)=44 sizeof(Parcel)=116
tharun@slotB:~/c_assignment$ gcc -Wall -Wextra task2.c -o t && ./t
502 1.2 kg Chennai-600077 12/10/2026
503 2.8 kg Coimbatore-641001 13/10/2026
501 4.5 kg Madurai-625001 12/10/2026
bytes copied per swap = 3 x sizeof(struct parcel) = 3 x 44 = 132
sizeof nested = 44, sizeof flat = 44
```

Analysis / Discussion

The first program shows that both access paths give the same PIN code (641001) and that a nested `Parcel` is 116 bytes. The second program prints the parcels in order of weight (1.2, 2.8, 4.5 kg), exactly as the expected output.

Alternative concept and complexity analysis

Cost of sorting: selection sort makes $O(n^2)$ comparisons and at most $n - 1$ swaps. Each swap copies the whole structure three times (temporary, then two assignments): $3 \times$ `sizeof(struct parcel)` $= 3 \times 44 = 132$ bytes, as measured.

Flat structure instead of nested: if city, pincode, day, month and year were written directly inside the parcel, `sizeof` is also 44 bytes, so nesting costs no extra space here (all members are 4-byte aligned and the layout is identical). The access path becomes shorter (`p->pincode` instead of `p->to.pincode`), but the address type cannot be reused: sender and receiver would need duplicated fields such as `s_city` and `r_city`, one date or address could not be passed to a function on its own (for example `print_address(&p->to)`), and a change to the address would have to be made in every copy. Nested structures give a slightly longer path but better reuse and clearer code.

Result

The entities were identified, the nested structures and the array for the day's bookings were defined, the pseudocode was written and the program prints the sorted parcels exactly as expected. I learnt how to build the access path for nested members with both the dot and arrow operators.

TASK 3 — Unions and Memory Layout — Weather-Station Sensor Packets

Task / Problem Statement

A weather station sends packets that carry exactly one kind of value: a temperature (float), a humidity percentage (int) or a short status text (up to 7 characters). The station has very little memory and stores thousands of packets. The task is to (3.1) design the packet as a structure with all three fields and as a tagged union and justify the choice, (3.2) predict and verify `sizeof` for each design, (3.3) implement the tagged union with a print function that uses the tag, and (3.4) run a misuse experiment that stores a temperature and reads the humidity member. TODO 1 of the given program is completed.

Aim

To use a tagged union so that one block of memory holds one of several value types safely, and to understand size, alignment and padding of structures and unions.

Concept / Theory

A **union** keeps all its members at the same address, so its size is that of the largest member (rounded up to the strictest alignment). Only the member last written is valid. A **tagged union** is a structure holding an enumeration **tag** that records which member is valid, plus the union; a `switch` on the tag reads the right member. **Alignment:** a type of size n is usually placed at an address that is a multiple of its alignment (4 for `int` and `float`); the compiler inserts **padding** and rounds the total size to a multiple of the largest alignment.

Procedure / Algorithm

1. **Design A (3.1):** a structure with a tag, `float celsius`, `int percent` and `char status[8]` — all three fields exist at once. **Design B:** a tag plus `union { float celsius; int percent; char status[8]; }`. The station stores thousands of packets and only one field is ever valid, so Design B is the right choice: it needs 12 bytes instead of 20 (a saving of 8 bytes, 40 %, per packet; for 10,000 packets 120,000 bytes instead of 200,000).
2. **Predict (3.2):** union = largest member `char[8]` = 8 bytes, alignment 4, so size 8. `struct reading` = tag 4 bytes at offset 0 + union (alignment 4) at offset 4 with 8 bytes = 12 bytes, a multiple of 4, no padding. Design A = 4 + 4 + 4 + 8 = 20 bytes (16 without the tag).
3. Write `enum kind`, `struct reading` and `struct packet_struct` and print all `sizeof` values to verify the prediction.
4. Write `show()` using a `switch` on `r->type` to print only the valid member (TODO 1).
5. Create the three sample packets with designated initialisers and print them.
6. **Misuse (3.4):** store 31.5 in `celsius`, read `percent`, print it in decimal and hexadecimal.

Program / Source Code

task3.c

```
#include <stdio.h>
#include <string.h>

enum kind { TEMPERATURE, HUMIDITY, STATUS };

/* Design A: plain structure holding all three fields */
struct packet_struct { enum kind type; float celsius; int percent; char status[8]; };

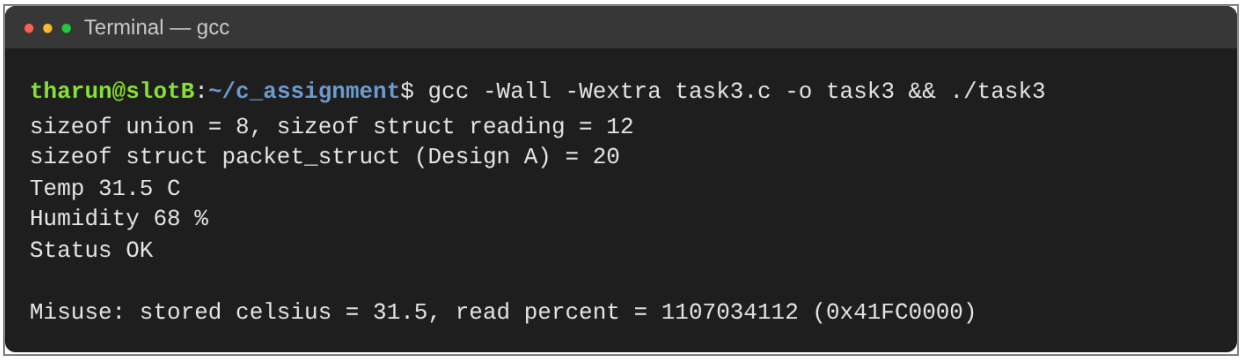
/* Design B: tagged union */
struct reading {
    enum kind type;                 /* tag: tells which union member is valid */
    union {
        float celsius;
        int   percent;
        char  status[8];
    } value;
};
```

```
void show(const struct reading *r)
{
    switch (r->type) {
    case TEMPERATURE: printf("Temp %.1f C\n",   r->value.celsius); break;
    case HUMIDITY:    printf("Humidity %d %%\n", r->value.percent); break;
    case STATUS:      printf("Status %s\n",      r->value.status);  break;
    }
}

int main(void)
{
    struct reading log[3] = {
        {TEMPERATURE, {.celsius = 31.5f}},
        {HUMIDITY,    {.percent = 68}},
        {STATUS,      {.status = "OK"}}
    };
    printf("sizeof union = %zu, sizeof struct reading = %zu\n",
           sizeof log[0].value, sizeof log[0]);
    printf("sizeof struct packet_struct (Design A) = %zu\n", sizeof(struct packet_struct));
    for (int i = 0; i < 3; i++) show(&log[i]);

    /* Task 3.4 - misuse experiment */
    struct reading m;
    m.type = TEMPERATURE;
    m.value.celsius = 31.5f;
    printf("\nMisuse: stored celsius = %.1f, read percent = %d (0x%08X)\n",
           m.value.celsius, m.value.percent, (unsigned)m.value.percent);
    return 0;
}
```

Output Screenshot



Analysis / Discussion

The measured sizes (union 8, `struct reading` 12, Design A 20) match the prediction. `struct reading` is 12 bytes and not 4 + 4 + 4 + 8 = 20 because `celsius`, `percent` and `status` are stored in the same 8 bytes; only the 4-byte tag is added to the 8-byte union.

Misuse experiment: after storing 31.5 in `celsius`, reading `percent` printed **1107034112** (`0x41FC0000`). The union does not convert the value; it reinterprets the same four bytes. In IEEE-754 these bytes (sign 0, exponent 10000011, fraction 11111 followed by zeros) represent $1.96875 \times 2^4 = 31.5$, but as an `int` they read as 1107034112. This is why the tag must always be checked before a member is read.

Alternative concept and complexity analysis

Three separate arrays (one per type): each value needs only its own size (4, 4 or 8 bytes) and no tag, but the arrival order of the packets is lost unless an extra index or timestamp array (about 4 bytes per packet) is kept, and the code must know which array to read. Reading a known packet is O(1).

malloc and a `void *` per packet: reading a packet is still O(1), but needs a pointer dereference to a separately allocated block (poor cache locality) and a way to know its type, so a tag is still needed. Each packet costs an 8-byte pointer plus the allocated block, whose size on a typical 64-bit allocator is rounded up with bookkeeping

overhead (commonly 16–32 bytes for a small block), and every block must be freed. For thousands of packets this uses several times more memory than the 12-byte tagged union.

Tagged union: 12 bytes per packet, contiguous, one allocation, O(1) read with a `switch` — the best fit for a memory-poor station.

Result

The tagged-union design was implemented, the predicted sizes (8, 12 and 20 bytes) matched the measured sizes, `show()` prints only the valid member, and the misuse experiment was explained. I learnt that a union saves memory but needs a tag, because reading the wrong member only reinterprets the bytes.

TASK 4 — Structure Forensics — Passing Structures to Functions

Task / Problem Statement

A bank's savings-account program is supposed to add interest to every account. After running it, every balance is unchanged and one statement does not compile. The task is to (4.1) locate the two defects, (4.2) print the address of the structure inside `addinterest()` and in `main()` as evidence, (4.3) repair the program with a structure pointer and the arrow operator, and (4.4) explain when a structure should be passed by value, by pointer and by const pointer.

Aim

To find why a function does not change a structure that is passed by value, to prove it with addresses, and to repair it with a structure pointer.

Concept / Theory

C passes arguments **by value**: when a structure is passed, the function receives a **copy** stored at a different address, so changes to the copy never reach the caller. To modify the caller's structure the function receives its **address** (a structure pointer) and uses the arrow operator `->`. The dot operator works on a structure variable, the arrow operator on a pointer to a structure (`p->x` is `(*p).x`). A **const pointer** allows reading without copying and makes the compiler reject any change.

Procedure / Algorithm

Task 4.1 — Defects

Defect 1 (balances unchanged): `addinterest(struct account a, …)` receives a copy of `list[i]`. The interest is added to the copy, which disappears when the function returns, so the original is never changed.

Defect 2 (does not compile): `p` is a pointer, so `p.number` and `p.balance` use the dot operator on a pointer. gcc reports `'p' is a pointer; did you mean to use '->'?`. It must be `p->number` and `p->balance`. (The program also lacks `#include <stdio.h>` for `printf`.)

1. Compile the original program and read the compiler message for defect 2.
2. Write `evidence.c` that prints `&a` inside `addinterest()` and `&list[i]` in `main()` (Task 4.2).
3. Change the parameter to `struct account *a`, use `a->balance`, and call `addinterest(&list[i], 4)` (Task 4.3).
4. Print the accounts through the pointer with `p->number` and `p->balance`, using a const pointer in the print function.
5. Add the by-value-and-return version for comparison and print `sizeof` of the structure and of a pointer.

Program / Source Code

buggy.c — original program (does not compile)

```
struct account { int number; float balance; };
void addinterest(struct account a, float rate) {
    a.balance = a.balance + a.balance * rate / 100;
}
int main(void) {
    struct account list[3] = {{101, 5000}, {102, 12000}, {103, 800}};
    struct account *p = list;
    for (int i = 0; i < 3; i++) addinterest(list[i], 4);
    printf("%d %f\n", p.number, p.balance);
}
```

evidence.c — Task 4.2 evidence

```
#include <stdio.h>
struct account { int number; float balance; };
```

```
void addinterest(struct account a, float rate)          /* by value: gets a COPY */
{
    a.balance = a.balance + a.balance * rate / 100;
    printf("  inside addinterest: &a        = %p, balance = %.2f\n", (void *)&a, a.balance);
}

int main(void)
{
    struct account list[3] = {{101, 5000}, {102, 12000}, {103, 800}};
    for (int i = 0; i < 3; i++) {
        printf("main: &list[%d] = %p, balance before = %.2f\n", i, (void *)&list[i], list[i].balance);
        addinterest(list[i], 4);
        printf("main: balance after  = %.2f  (unchanged)\n", list[i].balance);
    }
    return 0;
}
```

task4.c — repaired program

```
#include <stdio.h>

struct account { int number; float balance; };

/* by pointer: modifies the caller's structure */
void addinterest(struct account *a, float rate)
{
    a->balance = a->balance + a->balance * rate / 100;
}

/* alternative: by value, return the modified copy */
struct account addinterest_val(struct account a, float rate)
{
    a.balance = a.balance + a.balance * rate / 100;
    return a;
}

/* read-only access: const pointer */
void print_account(const struct account *a)
{
    printf("%d %.2f\n", a->number, a->balance);
}

int main(void)
{
    struct account list[3] = {{101, 5000}, {102, 12000}, {103, 800}};
    struct account *p = list;

    for (int i = 0; i < 3; i++) addinterest(&list[i], 4);
    printf("By pointer:\n");
    for (p = list; p < list + 3; p++) print_account(p);   /* p->number, p->balance */

    struct account copy[3] = {{101, 5000}, {102, 12000}, {103, 800}};
    for (int i = 0; i < 3; i++) copy[i] = addinterest_val(copy[i], 4);
    printf("By value + return:\n");
    for (int i = 0; i < 3; i++) print_account(&copy[i]);

    printf("sizeof(struct account) = %zu bytes; pointer = %zu bytes\n",
           sizeof(struct account), sizeof(struct account *));
    return 0;
}
```

Output Screenshot

Terminal — gcc

```
tharun@slotB:~/c_assignment$ gcc buggy.c -o buggy 2>&1 | grep -E "error"
buggy.c:9:24: error: ‘p’ is a pointer; did you mean to use ‘->’?
buggy.c:9:34: error: ‘p’ is a pointer; did you mean to use ‘->’?
```

```
Terminal — gcc

tharun@slotB:~/c_assignment$ gcc -Wall evidence.c -o evidence && ./evidence
main: &list[0] = 0x7ffff4808b50, balance before = 5000.00
  inside addinterest: &a        = 0x7ffff4808b28, balance = 5200.00
main: balance after  = 5000.00  (unchanged)
main: &list[1] = 0x7ffff4808b58, balance before = 12000.00
  inside addinterest: &a        = 0x7ffff4808b28, balance = 12480.00
main: balance after  = 12000.00  (unchanged)
main: &list[2] = 0x7ffff4808b60, balance before = 800.00
  inside addinterest: &a        = 0x7ffff4808b28, balance = 832.00
main: balance after  = 800.00  (unchanged)
```

```
Terminal — gcc

tharun@slotB:~/c_assignment$ gcc -Wall -Wextra task4.c -o task4 && ./task4
By pointer:
101 5200.00
102 12480.00
103 832.00
By value + return:
101 5200.00
102 12480.00
103 832.00
sizeof(struct account) = 8 bytes; pointer = 8 bytes
```

Analysis / Discussion

Evidence (4.2): in `main()` the three accounts are at different addresses (…d8e0, …d8e8, …d8f0), while inside `addinterest()` the structure is always at …d8b8. This different address proves that the function works on a copy in its own stack frame: the copy gets 5200.00, 12480.00 and 832.00, but the balance in `main()` stays 5000, 12000 and 800. (Addresses change on every run because of address-space randomisation.)

Repair (4.3): with a pointer the function changes the original, and the balances become 5200.00, 12480.00 and 832.00 (4 % interest).

Task 4.4 — When to pass a structure how

Method	Use when	Example
By value	The structure is small, the function only needs a private copy, or it returns a new value.	`struct date d` (12 bytes) to compute a next day
By pointer	The function must modify the caller's structure, or the structure is large and copying is wasteful.	`addinterest(struct account *a, …)`
By const pointer	The function only reads a large structure; no copy is made and the compiler blocks accidental changes.	`print_account(const struct account *a)`

Alternative concept and complexity analysis

By value and return (`list[i] = addinterest(list[i], 4)`): it works too (second output block). Time is O(1) either way, but a call copies `sizeof(struct account)` bytes twice (in and out), against one 8-byte pointer. Here the structure is 8 bytes (the same as a pointer, measured), so there is no real difference. For a larger structure the cost grows: a 68-byte patient record would copy 136 bytes per call against 8 bytes for a pointer. Stack space follows the same pattern: the by-value version needs the structure's size in the callee's frame (plus the returned copy), the pointer version only 8 bytes. The by-value style also depends on the caller remembering to assign the result; forgetting it is the same bug again.

Result

Both defects were located (copy passed by value, and the dot operator used on a pointer), the different addresses proved the first one, and the repaired program adds interest to every account using a structure pointer and the

arrow operator. I learnt a simple rule: pass by pointer to modify, by const pointer to only read, and by value for small private copies.

TASK 5 — Storage Classes — The Same Counter Four Ways

Task / Problem Statement

A hostel mess issues token numbers starting from 1001, and the token function is called from more than one source file. The task is to (5.1) implement the token counter using (a) an automatic local variable, (b) a static local variable, (c) a global variable shared across two files with `extern`, and (d) a register loop variable in the printing loop, and record the tokens of each; (5.2) change the requirement to two counters (veg and non-veg) and record how much code changed; (5.3) compare the four storage classes and explain why the address of a register variable cannot be taken. TODO 1 (static local variable) of the given program is completed.

Aim

To compare automatic, static, global (extern) and register storage classes by building the same token counter with each, and to evaluate which one suits a counter that must keep its value.

Concept / Theory

auto: the default for local variables; created at every entry to the block, destroyed at exit, so it forgets its value. **static local:** created once before the program runs, keeps its value between calls, visible only inside its function. **global with extern:** defined once in one file; other files declare it with `extern` to share it; lives for the whole program and can be changed from anywhere. **register:** asks the compiler to keep a frequently used variable in a CPU register; it behaves like an `auto` variable but its address cannot be taken. C also does not specify the order in which the arguments of a function call are evaluated.

Procedure / Algorithm

1. **(a) auto:** `int token = 1000; return ++token;` — the variable is re-created at each call.
2. **(b) static:** `static int token = 1000; return ++token;` (TODO 1) — initialised once, value kept.
3. **(c) global:** define `int mess_counter = 1000;` in `globals.c`; in the other file declare `extern int mess_counter;` and use `++mess_counter`.
4. **(d) register:** print further tokens in a loop whose counter is `register int i`; also try to take `&i` to see the compiler's answer.
5. Call `printf("%d %d %d", next_token(), next_token(), next_token())` once to see the order of evaluation.
6. **5.2:** rewrite each version for two counters (veg, non-veg) and count the changed lines with a line-by-line diff of each version (`difflib`).
7. **Alternative:** keep the counter in the caller and pass `int *counter`.

Program / Source Code

task5.c — Task 5.1 (a), (b), (c), (d)

```
#include <stdio.h>

/* (a) automatic local variable: re-created on every call */
int auto_token(void)
{
    int token = 1000;
    return ++token;
}

/* (b) static local variable: created once, keeps its value */
int next_token(void)
{
    static int token = 1000;
    return ++token;
}

/* (c) global variable defined in globals.c, shared with extern */
extern int mess_counter;
```

```
int global_token(void)
{
    return ++mess_counter;
}

int main(void)
{
    int a1 = auto_token(),   a2 = auto_token(),   a3 = auto_token();
    int s1 = next_token(),   s2 = next_token(),   s3 = next_token();
    int g1 = global_token(), g2 = global_token(), g3 = global_token();
    printf("(a) auto   : %d %d %d\n", a1, a2, a3);
    printf("(b) static : %d %d %d\n", s1, s2, s3);
    printf("(c) global : %d %d %d\n", g1, g2, g3);

    /* (d) register loop variable in the printing loop */
    printf("(d) register loop (static counter continues): ");
    for (register int i = 0; i < 3; i++) printf("%d ", next_token());
    printf("\n");
    /* printf("%p", (void *)&i);  -> compile error: address of register variable */

    /* order of evaluation experiment */
    printf("single printf : %d %d %d\n", next_token(), next_token(), next_token());
    return 0;
}
```

globals.c — second source file

```
/* file 2 : owns the shared counter */
int mess_counter = 1000;
```

task5_v2.c — Task 5.2, two counters

```
#include <stdio.h>
enum { VEG, NONVEG };

/* (a) automatic: still re-created on every call, so both counters stay stuck */
int auto_token(int type)
{
    int token = 1000;
    return ++token;
}

/* (b) static: one array with a slot per counter */
int next_token(int type)
{
    static int token[2] = {1000, 1000};
    return ++token[type];
}

/* (c) global: two globals in globals2.c, shared with extern */
extern int veg_counter, nonveg_counter;
int global_token(int type)
{
    return type == VEG ? ++veg_counter : ++nonveg_counter;
}

int main(void)
{
    printf("(a) auto   veg: %d %d  nonveg: %d %d\n",
           auto_token(VEG), auto_token(VEG), auto_token(NONVEG), auto_token(NONVEG));
    int v1 = next_token(VEG), v2 = next_token(VEG), n1 = next_token(NONVEG), n2 = next_token(NONVEG);
    printf("(b) static veg: %d %d  nonveg: %d %d\n", v1, v2, n1, n2);
    int gv1 = global_token(VEG), gv2 = global_token(VEG), gn1 = global_token(NONVEG), gn2 =
global_token(NONVEG);
    printf("(c) global veg: %d %d  nonveg: %d %d\n", gv1, gv2, gn1, gn2);
    printf("(d) register loops: veg ");
    for (register int i = 0; i < 2; i++) printf("%d ", next_token(VEG));
    printf(" nonveg ");
    for (register int i = 0; i < 2; i++) printf("%d ", next_token(NONVEG));
    printf("\n");
```

```
    return 0;
}
```

globals2.c

```
/* file 2 : owns the two shared counters */
int veg_counter    = 1000;
int nonveg_counter = 1000;
```

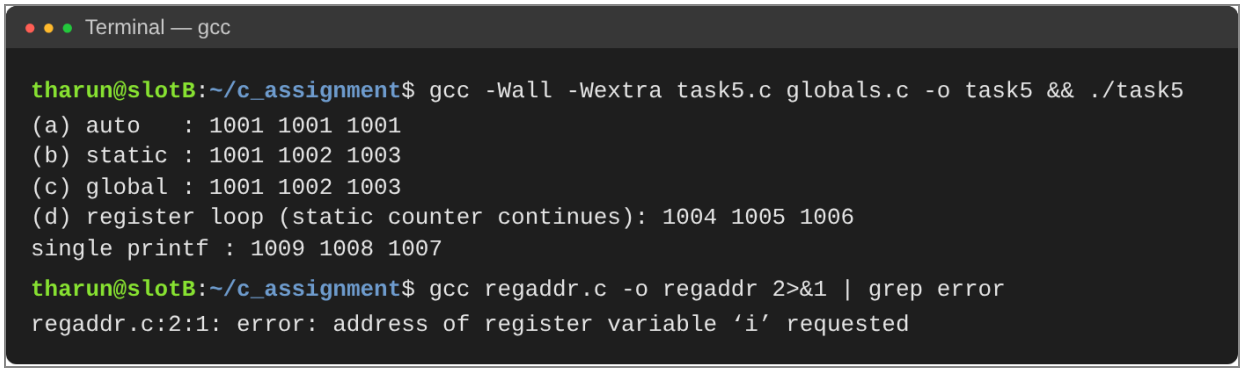
ptr.c — alternative: counter kept in the caller

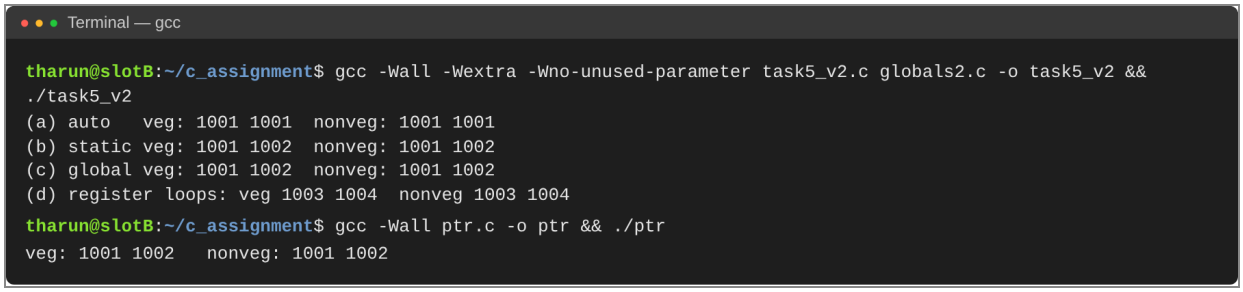
```
#include <stdio.h>

int next_token(int *counter)       /* counter lives in the caller */
{
    return ++*counter;
}

int main(void)
{
    int veg = 1000, nonveg = 1000;
    int v1 = next_token(&veg), v2 = next_token(&veg);
    int n1 = next_token(&nonveg), n2 = next_token(&nonveg);
    printf("veg: %d %d   nonveg: %d %d\n", v1, v2, n1, n2);
    return 0;
}
```

Output Screenshot





Analysis / Discussion

Task 5.1 outputs: (a) auto gives 1001 1001 1001 because the variable is re-created; (b) static gives 1001 1002 1003, which is the expected output; (c) global gives 1001 1002 1003; (d) the register loop printed 1004 1005 1006, continuing the static counter. Taking `&i` of a register variable gives `error: address of register variable 'i' requested`.

Order of evaluation: one `printf` with three `next_token()` calls printed **1009 1008 1007** on this gcc, i.e. the arguments were evaluated right to left. The C standard does not specify the order, so another compiler could print 1007 1008 1009; the result must not be relied on. The fix is to store each call in its own variable first, as done above.

Task 5.2 — Two counters: change in code

Version	Result with two counters	Lines changed (added + removed)
(a) auto	Still 1001 for both counters; cannot work — the variable is lost at every call	4 (even after the change it does not work)
(b) static	Works: `static int token[2]` with one slot per counter	8
(c) global	Works: two globals in `globals2.c` and a selecting function	13 (in two files)
(d) register	Only the loops change: one loop per counter	6

Task 5.3 — Comparison of the storage classes

Class	Scope	Lifetime	Initial value (if not set)	Risk of accidental modification
auto	The block	Until the block ends (each call)	Indeterminate (garbage)	Low — private, but the value is lost
static local	The function	Whole program	0	Low — only its own function can change it
global (extern)	All files that declare it	Whole program	0	High — any file can change it
register	The block	Until the block ends	Indeterminate (garbage)	Low — private; address cannot be taken

Why the address of a register variable cannot be taken: a register variable may be kept in a CPU register, and registers do not have memory addresses. The language therefore forbids `&` on it, even if the compiler finally decides to keep it in memory, and gcc gives the error shown above.

Alternative concept and complexity analysis

Counter in the caller, passed as `int *counter`: the output is `veg: 1001 1002 nonveg: 1001 1002`. **Time** is O(1) in all cases (one increment). **Space:** static and global counters occupy one `int` in static memory for the whole program; the caller's counter occupies one `int` on the caller's stack, living as long as the caller's block, plus an 8-byte pointer passed per call. **Independent counters:** with static or global storage a new counter needs a code change (array or another variable, 8 to 13 lines here); with the pointer, a new counter is just another variable in the caller and the function is not changed at all. The pointer version also has no hidden state, which makes it easier to test, but the caller must keep the counter alive and pass the right one.

Result

The four versions were implemented and gave the recorded tokens; the static and global versions keep the count, the auto version does not, and the register variable's address cannot be taken. The two-counter change showed that static needs a small change, global a larger one, and a pointer-based counter none. I learnt that the storage class decides scope, lifetime and how safely a counter can be shared.

TASK 6 — Design Challenge — College Canteen Inventory and Billing

Task / Problem Statement

The college canteen sells packaged items (with a price and expiry date), cooked items (with a price and the time they were prepared) and combo meals (made of up to three other items). It needs to record sales, update stock, warn about items expiring within two days, and print the day's bill summary sorted by revenue. The design is not given. The task is to go through five stages: (1) interpret the problem, (2) propose the data model, (3) identify the critical design decisions, (4) design the structures and functions, and (5) implement and demonstrate normal sales, a sale that exceeds stock, an expiring item and the sorted summary.

Aim

To design a complete data model from a written problem using nested structures, an array of structures and a tagged union, and to implement and test it.

Concept / Theory

Stage 1 — Interpret the problem

Entities and data: Item (id, name, kind, price, stock, units sold, revenue); **Packaged item** data (expiry date); **Cooked item** data (preparation time); **Combo meal** data (up to three part ids and how many); **Date** (day, month, year); **Time** (hour, minute); **Canteen** (the array of items, item count, today's date). A **sale** is an operation (item id and quantity).

Testable requirements:

R1. Every item has a unique id, a name and a price greater than 0.

R2. A packaged item has an expiry date, a cooked item a preparation time, and a combo has 1 to 3 part items.

R3. A sale with an unknown id or a quantity ≤ 0 is rejected.

R4. A sale is accepted only if stock ≥ quantity; stock never becomes negative, and a rejected sale changes nothing.

R5. An accepted sale reduces stock by the quantity, increases units sold, and adds quantity × price to revenue.

R6. A combo sale needs every part in stock for the quantity; it deducts stock from each part and records revenue at the combo price.

R7. A packaged item with 0 to 2 days left before expiry gets a WARNING; an item already past its date is reported EXPIRED.

R8. The summary lists all items in descending order of revenue and its total equals the sum of the item revenues.

Stage 2 — Data model

An **array of structures** (`Item items[MAX_ITEMS]` inside `Canteen`) stores all items. A **tagged union** is needed because each item is exactly one of three kinds with different extra data: the tag `Kind` says which of `packaged`, `cooked` or `combo` is valid, and the union (16 bytes, measured) is shared by the three. **Nested structures** are needed for the `Date` (expiry) and the `Time` (preparation), and a small array of three ids is nested for the combo.

Stage 3 — Critical design decisions

Safe stock update: check first, then deduct, in one function (`record_sale`) that returns a status code; for a combo, all parts are checked before any stock is deducted, so a failed sale never leaves half-updated stock. **Expiry check with a date structure:** each `Date` is converted to a day number (with leap years) and the difference from today's date is compared with 2; this avoids comparing day, month and year separately. **Static or global data:** only constants need to be fixed — the month-day table is a `static const` array in `day_number()` (created once), and `MAX_ITEMS` and `WARN_DAYS` are compile-time constants. Nothing that changes is global: the items, count and today's date are kept in one `Canteen` structure that is passed by pointer.

Procedure / Algorithm

Stage 4 — Function prototypes (structure pointers)

```
Item *find_item(Canteen *c, int id);
int   add_item(Canteen *c, const Item *it);
int   record_sale(Canteen *c, int id, int qty);      /* 0 ok, -1 not found, -2 no stock, -3 bad qty
*/
void  expiry_warning(const Canteen *c);
void  summary_by_revenue(const Canteen *c);
```

Pseudocode — recording a sale

```
record_sale(c, id, qty)
  item = find_item(c, id)
  IF item not found THEN RETURN NOT_FOUND
  IF qty <= 0 THEN RETURN BAD_QTY
  IF item is COMBO THEN
      FOR each part of the combo: IF part missing OR part.stock < qty THEN RETURN NO_STOCK
      FOR each part of the combo: part.stock = part.stock - qty
  ELSE
      IF item.stock < qty THEN RETURN NO_STOCK
      item.stock = item.stock - qty
  item.sold = item.sold + qty
  item.revenue = item.revenue + qty * item.price
  RETURN OK
```

Pseudocode — expiry warning

```
expiry_warning(c)
  today = day_number(c.today); found = 0
  FOR each item in c
      IF item is PACKAGED THEN
          left = day_number(item.expiry) - today
          IF left < 0 THEN print "EXPIRED" ; found = 1
          ELSE IF left <= 2 THEN print "WARNING", left ; found = 1
  IF found = 0 THEN print "No item expires within 2 days"
```

Stage 5 — Implementation steps

1. Define `Date`, `Time`, `Kind`, `Item` (with the tagged union) and `Canteen`.
2. Write `day_number()`, `find_item()`, `add_item()`, `record_sale()`, `expiry_warning()` and `summary_by_revenue()` (sorts an array of pointers, not whole structures).
3. In `main()` set today's date to 11/11/2026 and add five items: Biscuits and Chips (packaged), Samosa and Tea (cooked) and a Snack Combo (Samosa + Tea + Biscuits).
4. Demonstrate normal sales, a sale that exceeds stock, an unknown item, the expiry warning and the sorted summary.

Program / Source Code

task6.c

```
#include <stdio.h>
#include <string.h>

#define MAX_ITEMS 20
#define WARN_DAYS 2

typedef struct { int day, month, year; } Date;
typedef struct { int hour, minute; } Time;
typedef enum { PACKAGED, COOKED, COMBO } Kind;

typedef struct {
    int   id;
```

```
    char  name[24];
    Kind  kind;                         /* tag */
    float price;
    int   stock;
    int   sold;
    float revenue;
    union {                             /* only one member is valid */
        struct { Date expiry; }                packaged;
        struct { Time prepared; }              cooked;
        struct { int part_id[3]; int nparts; } combo;
    } u;
} Item;

typedef struct {
    Item items[MAX_ITEMS];
    int  count;
    Date today;
} Canteen;

enum { SALE_OK = 0, SALE_NOT_FOUND = -1, SALE_NO_STOCK = -2, SALE_BAD_QTY = -3 };

/* ---------- date helper: days since 01/01/0001 (Gregorian) ---------- */
static int day_number(Date d)
{
    static const int cum[] = {0,31,59,90,120,151,181,212,243,273,304,334};
    int y = d.year - 1;
    int n = y*365 + y/4 - y/100 + y/400 + cum[d.month-1] + d.day;
    int leap = (d.year%4==0 && d.year%100!=0) || d.year%400==0;
    if (leap && d.month > 2) n++;
    return n;
}

/* ---------- operations ---------- */
Item *find_item(Canteen *c, int id)
{
    for (int i = 0; i < c->count; i++)
        if (c->items[i].id == id) return &c->items[i];
    return NULL;
}

int add_item(Canteen *c, const Item *it)
{
    if (c->count >= MAX_ITEMS) return -1;
    c->items[c->count++] = *it;
    return 0;
}

int record_sale(Canteen *c, int id, int qty)
{
    Item *it = find_item(c, id);
    if (!it)       return SALE_NOT_FOUND;
    if (qty <= 0)  return SALE_BAD_QTY;

    if (it->kind == COMBO) {                        /* check every part first */
        for (int k = 0; k < it->u.combo.nparts; k++) {
            Item *part = find_item(c, it->u.combo.part_id[k]);
            if (!part || part->stock < qty) return SALE_NO_STOCK;
        }
        for (int k = 0; k < it->u.combo.nparts; k++)  /* then deduct */
            find_item(c, it->u.combo.part_id[k])->stock -= qty;
    } else {
        if (it->stock < qty) return SALE_NO_STOCK;   /* never go negative */
        it->stock -= qty;
    }
    it->sold    += qty;
    it->revenue += qty * it->price;
    return SALE_OK;
}

void expiry_warning(const Canteen *c)
{
    int today = day_number(c->today), found = 0;
```

```
    for (int i = 0; i < c->count; i++) {
        const Item *it = &c->items[i];
        if (it->kind != PACKAGED) continue;
        int left = day_number(it->u.packaged.expiry) - today;
        if (left < 0)              { printf("EXPIRED : %-10s (expired %d day(s) ago)\n", it->name, -left);
found = 1; }
        else if (left <= WARN_DAYS){ printf("WARNING : %-10s expires in %d day(s) (%02d/%02d/%d)\n", it->name,
left,
                                           it->u.packaged.expiry.day, it->u.packaged.expiry.month, it-
>u.packaged.expiry.year); found = 1; }
    }
    if (!found) printf("No item expires within %d days.\n", WARN_DAYS);
}

void summary_by_revenue(const Canteen *c)
{
    const Item *order[MAX_ITEMS];                   /* sort pointers, not big structures */
    for (int i = 0; i < c->count; i++) order[i] = &c->items[i];
    for (int i = 0; i < c->count - 1; i++)
        for (int j = i + 1; j < c->count; j++)
            if (order[j]->revenue > order[i]->revenue) {
                const Item *t = order[i]; order[i] = order[j]; order[j] = t;
            }
    float total = 0;
    printf("%-4s %-12s %5s %9s\n", "ID", "Item", "Sold", "Revenue");
    for (int i = 0; i < c->count; i++) {
        printf("%-4d %-12s %5d %9.2f\n", order[i]->id, order[i]->name, order[i]->sold, order[i]->revenue);
        total += order[i]->revenue;
    }
    printf("%-22s %9.2f\n", "TOTAL", total);
}

static const char *msg(int r)
{
    switch (r) {
    case SALE_OK: return "OK";
    case SALE_NOT_FOUND: return "REJECTED - item not found";
    case SALE_NO_STOCK: return "REJECTED - insufficient stock";
    default: return "REJECTED - bad quantity";
    }
}

int main(void)
{
    Canteen c = { .count = 0, .today = {11, 11, 2026} };
    Item biscuit = {1, "Biscuits", PACKAGED, 10, 20, 0, 0, .u.packaged = {{12, 11, 2026}} };
    Item chips   = {2, "Chips",    PACKAGED, 20, 15, 0, 0, .u.packaged = {{30, 11, 2026}} };
    Item samosa  = {3, "Samosa",   COOKED,   12, 30, 0, 0, .u.cooked   = {{10, 30}} };
    Item tea     = {4, "Tea",      COOKED,   10, 50, 0, 0, .u.cooked   = {{9, 15}} };
    Item combo   = {5, "Snack Combo", COMBO, 30,  0, 0, 0, .u.combo    = {{3, 4, 1}, 3} };
    add_item(&c, &biscuit); add_item(&c, &chips); add_item(&c, &samosa);
    add_item(&c, &tea);     add_item(&c, &combo);

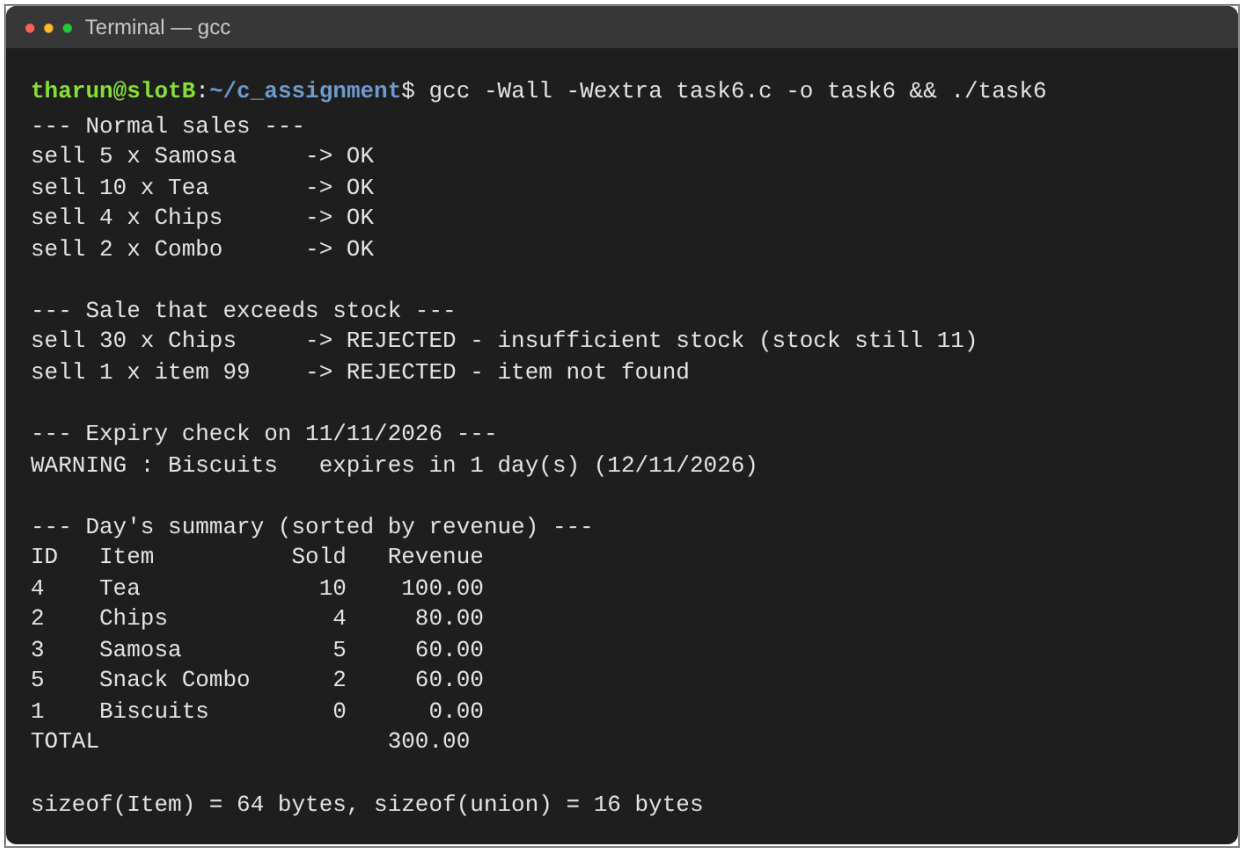
    printf("--- Normal sales ---\n");
    printf("sell 5 x Samosa     -> %s\n", msg(record_sale(&c, 3, 5)));
    printf("sell 10 x Tea       -> %s\n", msg(record_sale(&c, 4, 10)));
    printf("sell 4 x Chips      -> %s\n", msg(record_sale(&c, 2, 4)));
    printf("sell 2 x Combo      -> %s\n", msg(record_sale(&c, 5, 2)));

    printf("\n--- Sale that exceeds stock ---\n");
    printf("sell 30 x Chips     -> %s (stock still %d)\n", msg(record_sale(&c, 2, 30)), find_item(&c, 2)-
>stock);
    printf("sell 1 x item 99    -> %s\n", msg(record_sale(&c, 99, 1)));

    printf("\n--- Expiry check on 11/11/2026 ---\n");
    expiry_warning(&c);

    printf("\n--- Day's summary (sorted by revenue) ---\n");
    summary_by_revenue(&c);
    printf("\nsizeof(Item) = %zu bytes, sizeof(union) = %zu bytes\n", sizeof(Item), sizeof c.items[0].u);
    return 0;
}
```

Output Screenshot



Analysis / Discussion

Normal sales were accepted (5 Samosa, 10 Tea, 4 Chips and 2 Combos; the combos also deducted stock from Samosa, Tea and Biscuits). **The sale of 30 Chips was rejected** because only 15 − 4 = 11 were left, and the stock stayed 11. An unknown item was rejected. **Expiry:** on 11/11/2026 Biscuits (expiry 12/11/2026) get a WARNING for 1 day left, while Chips (30/11/2026) are not reported. **The summary** is in descending order of revenue (Tea 100, Chips 80, Samosa 60, Combo 60, Biscuits 0) and the total 300.00 equals the sum. All requirements R1 to R8 were exercised by these runs.

Alternative concept and complexity analysis

Linked list of dynamically allocated structures instead of the array: adding at the head is O(1) (the array appends at the end in O(1) while there is room). Removing an item costs O(n) to find it in the list, then O(1) to unlink it, while the array needs O(n) to find it and also O(n) to shift the later elements. Searching by id is O(n) in both (without sorting). The list has no fixed `MAX_ITEMS` limit and never shifts structures, but it needs an extra 8-byte `next` pointer per item plus allocator overhead for each `malloc`, gives no random access and has worse cache locality. The sorted summary would use a merge sort on the list (O(n log n)) or the same pointer-array approach.

Result

The canteen problem was analysed into entities and testable requirements, a data model with an array of structures, nested structures and a tagged union was designed, and the working program demonstrated normal sales, a rejected sale that exceeds stock, an expiry warning and the revenue-sorted summary. I learnt how to turn a problem statement into structures, functions and test cases.

Test Cases / Validation Table

Record different inputs, expected results, actual results, and whether each test passed.

Test No.	Input / Scenario	Expected Output	Actual Output	Status
1	Task 1: add a patient with phone “98765” (5 digits)	Rejected	add Anu -> REJECTED	Pass
2	Task 1: sort 54, 23, 67 year-old patients by age	Meena 23, Ravi 54, Kumar 67 with all fields together	Meena 23, Ravi 54, Kumar 67	Pass
3	Task 1: add to a register that already holds 100	Rejected (no overflow)	add to full register -> REJECTED	Pass
4	Task 2: sort parcels 501 (4.5 kg), 502 (1.2 kg), 503 (2.8 kg)	502, 503, 501 in that order	502 1.2 kg … / 503 2.8 kg … / 501 4.5 kg …	Pass
5	Task 2: PIN code of the 3rd parcel, array and pointer paths	Same value from both paths	641001 and 641001	Pass
6	Task 3: sizeof union, struct reading, packet_struct	8, 12, 20	8, 12, 20	Pass
7	Task 3: store 31.5 in celsius, read percent	Meaningless integer (bit pattern of 31.5)	1107034112 (0x41FC0000)	Pass
8	Task 4: add 4 % interest by value / by pointer	Unchanged / 5200, 12480, 832	5000 (unchanged) / 5200, 12480, 832	Pass
9	Task 5: three tokens with auto / static / global	1001 1001 1001 / 1001 1002 1003 / 1001 1002 1003	Same as expected	Pass
10	Task 5: take &i of a register variable	Compile error	error: address of register variable 'i' requested	Pass
11	Task 6: sell 30 Chips when 11 are in stock	Rejected, stock stays 11	REJECTED - insufficient stock (stock still 11)	Pass
12	Task 6: expiry check on 11/11/2026 (Biscuits expire 12/11/2026)	Warning for Biscuits only	WARNING : Biscuits expires in 1 day(s)	Pass
13	Task 6: summary sorted by revenue	Tea, Chips, Samosa / Combo, Biscuits; total 300.00	Tea 100, Chips 80, Samosa 60, Combo 60, Biscuits 0; TOTAL 300.00	Pass

Final Submission Checklist

☑ Student and course details completed

☑ Task statement, aim, theory, and algorithm included

☑ Program code is readable and runs successfully

☑ Output screenshot included

☑ Analysis and result written in own words

☑ Test cases completed where applicable

☐ GitHub repository link added if required